

Program 1 - Sorting

Parker Macomber

Part 1: For the assignment, the three sorting methods that I chose were Bubble-sort for $O(n^2)$, Merge-sort for $O(n \log n)$, and Counting-sort for $O(n)$.

Part 2 Pseudocodes:

Pseudocode Bubble sort:

```
Bubblesort(A)
1  for i = 1 to A.length - 1
2      for j = A.length down to i + 1
3          if A[j] < A[j - 1]
4              exchange A[j] with A[j - 1]
```

Source: Cormen T., Leiserson C., Rivest R., & Stein C. (2009). Introduction to Algorithms Third Edition. Cambridge: Massachusetts Institute of Technology. (page: 40)

Pseudocode Merge sort:

```
MergeSort(A,p,r)
1  if p < r
2      q = [(p+r)/2]
3      MergeSort(A,p,q)
4      MergeSort(A,q + 1,r)
5      Merge (A, p, q, r)
```

```
Merge (A, p, q, r)
1  n1 = q - p + 1
2  n2 = r - q
3  let L[1...n1 + 1] and R[1...n2 + 1] be new arrays
4  for i = 1 to n1
5      L[i] = A[p + i - 1]
6  for j = 1 to n2
7      R[j] = A[q + j]
8  L[n1 + 1] = infinite
9  R[n2 + 1] = infinite
10 i = 1
11 j = 1
12 for k = p to r
13     if L[i] <= R[j]
14         A[k] = L[i]
15         i = i + 1
```

```

16     else A[ k ] = R[ j ]
17         j = j + 1

```

Source: Cormen T., Leiserson C., Rivest R., & Stein C. (2009). Introduction to Algorithms Third Edition. Cambridge: Massachusetts Institute of Technology. (page: 31)

Pseudocode Counting Sort:

```

Counting-Sort( A, B, k )
1  let C[0...k] be a new array
2  for i = 0 to k
3      C[ i ] = 0
4  for j = 1 to A.length
5      C[ A[ j ] ] = C[ A[ j ] ] + 1
7  for i = 1 to k
8      C[ i ] = C[ i ] + C[ i - 1 ]
9  for j = A.length down to 1
10     B[ C[ A[ j ] ] ] = A[ j ]
11     C[ A[ j ] ] = C[ A[ j ] ] - 1

```

Source: Cormen T., Leiserson C., Rivest R., & Stein C. (2009). Introduction to Algorithms Third Edition. Cambridge: Massachusetts Institute of Technology. (page: 195).

Part 3:

Bubble-Sort loop invariant:

At the beginning of each iteration of the for loop in lines 1-4, all numbers have not been sorted or seen in the random number array.

Initialization: When the loop starts there are no sorted numbers so far.

Maintenance: When no numbers have been sorted. Then it takes the first number in the array and compares it until it has compared all numbers in the array, then it terminates.

Termination: The loop terminates when all numbers are sorted in ascending order.

Merge-Sort loop invariant:

At the beginning of each iteration of the for loop in lines 12-17, all numbers have not been sorted in the random number array.

Initialization: When entering the loop, the array in question has been evenly split into two different arrays. All of the numbers have yet to be sorted in either array.

Maintenance: When none of the numbers have been sorted. The loop compares the left array to the right array to figure out which number is smaller and then asserts that number into the array. It does this until all numbers in the array are sorted.

Termination: The loop terminates when all numbers have been sorted out in ascending order.

Count-Sort loop invariant:

At the beginning of each iteration of the for loop in lines 10-12, all numbers have not been sorted or seen in the random number array.

Initialization: When entering the loop, none of the numbers in the array are sorted.

Maintenance: When going through the loop, each element that is in array A, is then sorted into its correct position in the output array B. It does this by having C[A[j]] being the correct output position and adding the value of A[j] into it.

Termination: When all numbers have been gone through and sorted, the loop will terminate.

Part 4:

The language I will be using is Python.

Bubble-Sort code:

```
n_comparisons = 0
```

```
def main():
```

```
    filename = input("Enter a file: ") # Enter file input
```

```
    file = open(filename, 'r') # opening file and then reading it
```

```
    A2 = file.readlines()
```

```
    A2.pop(0) # get rid of the first line in text file
```

```
    bubbleSort(A2) # calling bubbleSort function
```

```
    print(A2) # printing out sorted array and amount of comparisons
```

```
    print (n_comparisons)
```

```
def compare_gt(x,y): # comparison function
```

```
    global n_comparisons
```

```
    n_comparisons +=1 # Whenever something is being compared add one to it
```

```
    return x > y # This will only return if x is greater than y
```

```
def bubbleSort(A): # bubbleSort function
```

```
    n = len(A) # variables
```

```
    for i in range(n-1): # for loop for going through the length of the array
```

```
        for j in range (n-i-1): # for loop for comparing numbers
```

```
if compare_gt(A[j], A[j+1]): # compaing number in the array to next number in the array
    A[j], A[j+1] = A[j+1], A[j] # Swapping numbers if A[j] is greater than A[j+1]
```

Source: Williams D. Program 1-Sorting example bubble-sort. (I wanted to add this since some of the bubble-sort code and comparison code was based on the example given to us).

Merge-Sort:

```
def main():
    filename = input("Enter a file: ") # Enter file input
    file = open(filename, 'r') # opening file and then reading it
    A = file.readlines()
    A.pop(0) # getting rid of first line in text file
    p = 0 # creating variables
    r = len(A)
    n_comparison = 0
    MergeSort(A,p,r-1) # calling merge sort function
    print("Sorted Array: ", A) # printing out sorted array and comparisons
    print("comparisons: ", n_comparison)

def MergeSort(A,p,r, n_comparisons):
    if p < r-1:
        q = (p+r)//2 # dividing length into two
        MergeSort(A,p,q,n_comparisons) # calling recursive function
        MergeSort(A,q + 1,r,n_comparisons)
        Merge(A,p,q,r,n_comparisons) # calling merge function

def Merge(A,p,q,r,n_comparisons):
    n1 = q - p + 1 # creating variables for size of left and right arrays
    n2 = r - q

    L = [0]*(n1) # creating left and right arrays.
    R = [0]*(n2)

    for i in range(0, n1):
        L[i] = A[p+i] # putting first half of the array into left array

    for j in range(0, n2):
        R[j] = A[q+1+j] # putting the second half of the array inot the right array

    i = 1 # creating variables
    j = 1

    for k in range(p, r): # looping through the length of the main array
```

```

if L[i] <= R[j]: # comparing left array to right array and if left is less than, then add it to main
    #array
    A[k] = L[i]
    i = i + 1
    N_compaision += 1 # adding a comparison
else:
    j = j + 1 # stating that if right is greater than, then continue the loop
    N_compaision += 1 # adding a comparison

```

Count-Sort:

```

def main():
    filename = input("Enter a file: ") # Enter file input
    file = open(filename,'r') #opening file and then reading it
    A1 = file.readlines()
    A1.pop(0) # getting rid of first line in text file
    k = max(A1)+1 # creating variables
    size1 = len(A1)
    B1 = [0]*size1
    n_comparisons = len(A1)

    CountSorts(A1,B1,k) #calling count-sort
    print("Sorted array: ",B1) #printing array and comparisons
    print ("comparisons: ", n_comparisons)

def CountSorts(A1,B1,k):
    C1 = [0] *k # creating empty list with k amount amount of elements all being 0
    size = len(A1) # creating size of array

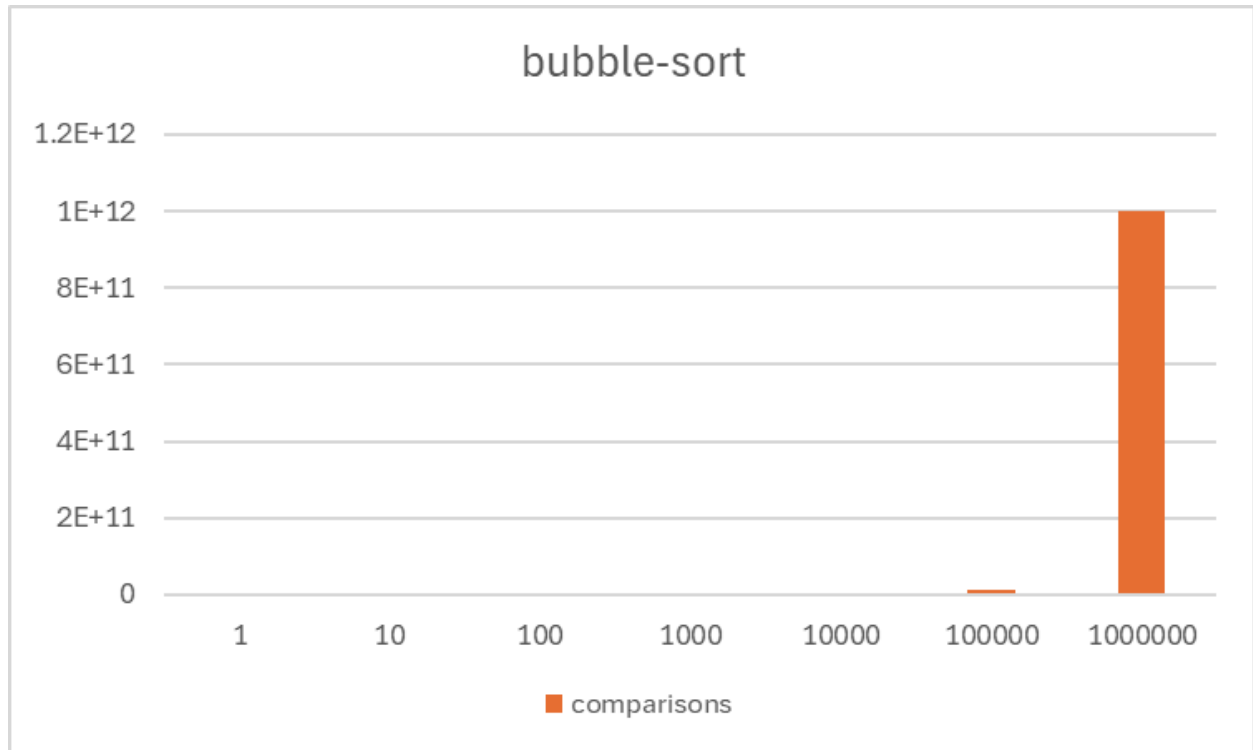
    for j in range (0,size):
        C1[A1[j]] += 1 # containing elements equal to i
    for i in range(1, k):
        C1[i] += C1[i-1] # containing the number of elements less than or equal to i

    for j in reversed(range(0, size)):
        B1[C1[A1[j]]-1] = A1[j]
        C1[A1[j]] = C1[A1[j]] - 1 # putting elements into correct order and into an array

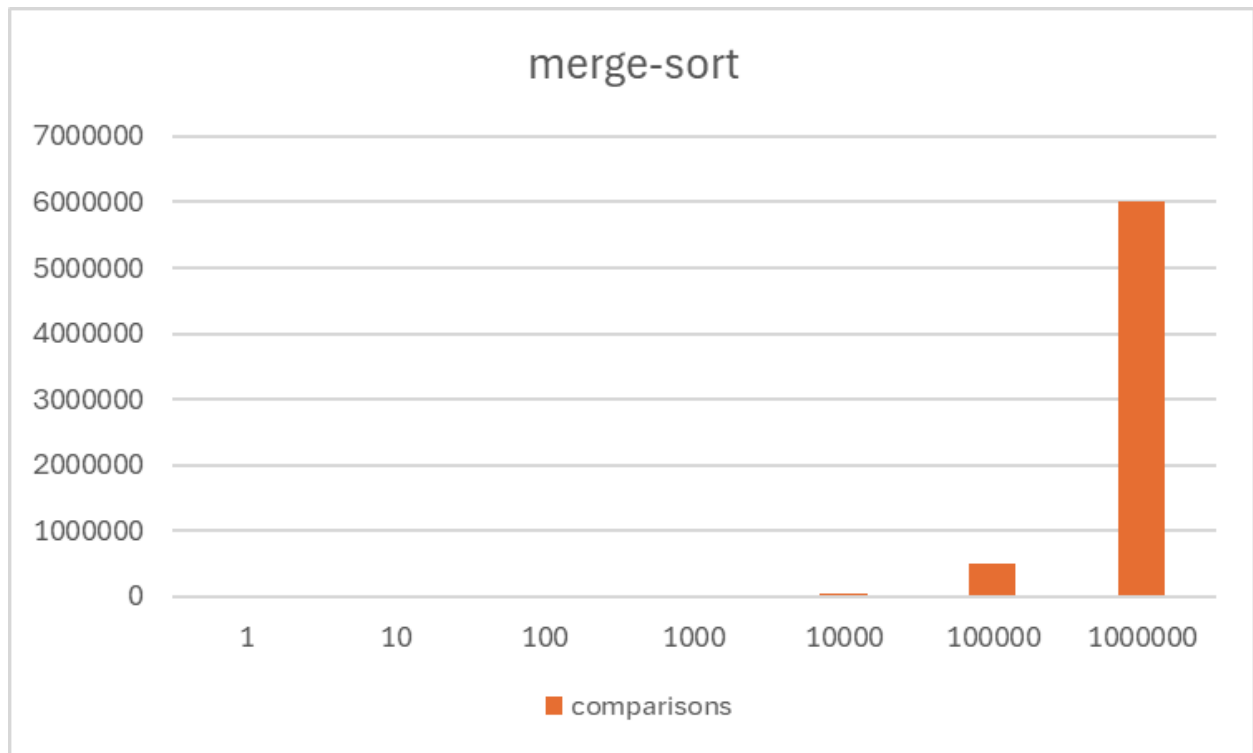
```

Part 5:

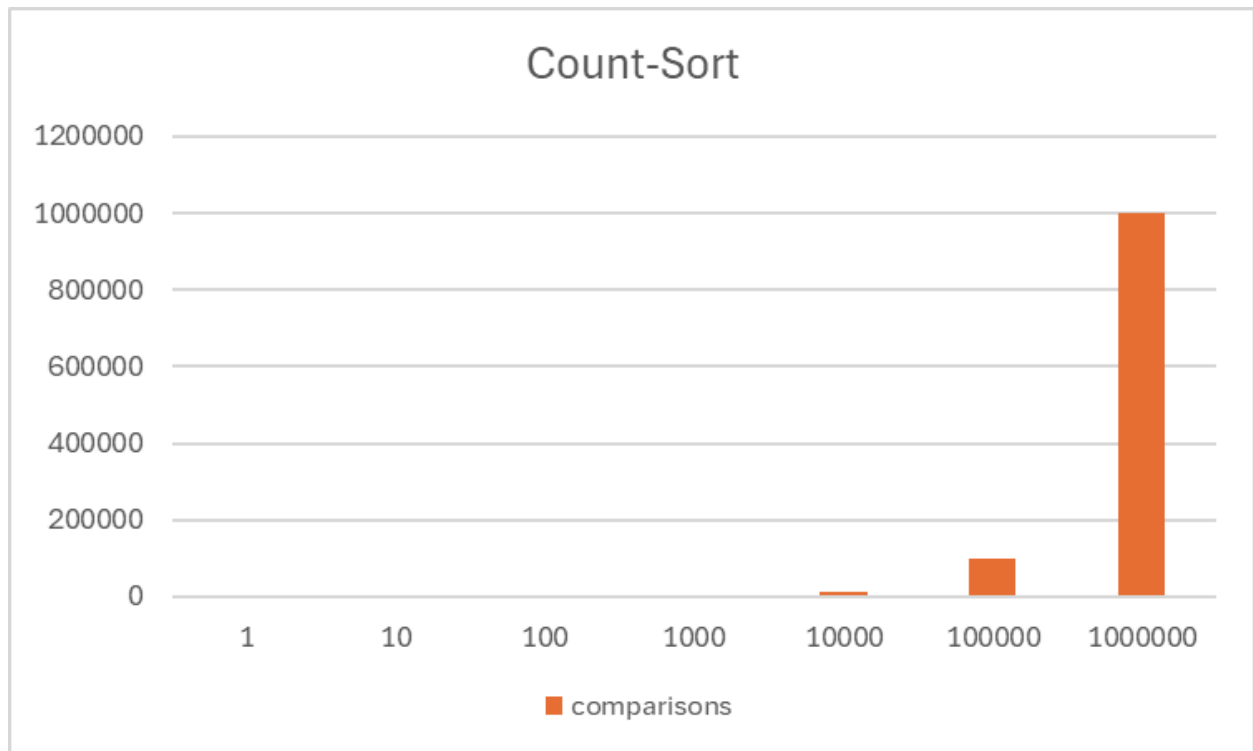
Bubble-Sort Analysis of Growth:



Merge-Sort Analysis of Growth:



Count-Sort Analysis of Growth



Does this meet the expectations of growth?

Yes, it does show my expectations in terms of growth because as seen in bubble-sort, we can determine that the rate of growth is growing in terms of $O(n^2)$. This is also seen in merge-sort where it does follow along the lines of $O(n \log n)$ because $(1000000 \log 1000000)$ is 6000000 and the graph does show us that it is roughly that amount. Then finally in count-sort it does follow the rate of growth of $O(n)$ because of how it does one million iterations on a million numbers, same with one hundred thousand numbers, it does that one hundred thousand times, and so on.

Comparison across data:

Shuffled: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 260 iterations, and bubble-sort was the worst going through 15453 iterations.

Sorted: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 200 iterations, and bubble-sort was the worst by going through 10000 iterations.

Nearly-Sorted: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 202 iterations, and bubble-sort was the worst by going through 10201 iterations.

Unsorted: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 202 iterations, and bubble-sort was the worst by going through 10302

Nearly-unsorted: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 352 iterations, and bubble-sort was the worst by going through 25755

Duplicates: Count-sort performed the best by only going through 100 iterations, Merge-sort was next by only going through 395 iterations, and bubble-sort was the worst by going through 30906